

Simulation

Installation:-

We will use Gazebo Fortress (aka Ignition Gazebo 6) as our simulation platform as it is compatible with ROS Humble.

Follow the [Fortress Installation](#) for complete installation in ubuntu.

Installation Check:-

Run the following command to check if fortress is properly installed.

```
Shell  
ign gazebo --version
```

It will print the version no. similar to 6.x.x

What is Gazebo?

Gazebo is a robotics simulator that provides a realistic physics environment for testing and developing robots. It allows users to simulate robot models, sensors, and environments before deploying them on real hardware. Gazebo uses the Simulation Description Format (SDF), an XML-based format used to describe worlds, models, links, joints, sensors, and other simulation components.

To simulate our model we need both urdf(robot description file) and a world file in our workspace. Following is a reference to a proper workspace

```
None  
ros2_ws/  
├─ src/  
│   └─ mini_bot/  
│       ├── CMakeLists.txt  
│       ├── package.xml  
│       ├── urdf/  
│       └─ mini_bot.urdf
```

```
|      |— worlds/
|      |   |— world.sdf
|      |— launch/
|      |   |— display.launch.py
|      |   |— gazebo.launch.py
|      |— scripts
```

You can launch your urdf or sdf files using command

Shell

```
ign gazebo world.sdf
```

Shell

```
ign gazebo mini_bot.sdf
```

However, as the complexity of a robotics application increases, manually launching each component becomes cumbersome. ROS 2 launch files provide a convenient way to automate this process. A launch file can automatically load the robot description (URDF/Xacro), simulation world (SDF), Gazebo plugins, controllers, and any additional ROS 2 nodes required for the application.

Using launch files simplifies the simulation workflow and allows an entire robotics system to be started with a single command, making them particularly useful for larger projects.

You can refer to [README](#) to see how the urdf and sdf files are defined.

SDF/World File

SDF (Simulation Description Format) is an XML-based format used by Gazebo to describe simulation environments and robot models. An SDF file can define a complete simulation world, including robots, objects, lights, sensors, physics properties, and plugins.

SDF Structure

The <sdf> tag is the root element of the file and specifies the SDF version being used.

World

The <world> tag defines the simulation environment. All objects, robots, lights, physics settings, and plugins are placed inside this tag.

Physics

The physics section controls how the simulation behaves.

- **max_step_size**: Simulation time step.
- **real_time_factor**: Desired simulation speed relative to real time.

Plugins

Plugins extend the functionality of the simulation.

Common plugins include: Physics plugin, User command plugin, Scene broadcaster plugin, DiffDrive plugin, and Sensor plugins.

Light

Lights illuminate the simulation world.

Common light types: Directional, Point, Spot.

Model

A model represents any object in the world.

Examples: Robots, Obstacles, Tables, Ground plane.

Link

A model is made up of one or more links.

A link can contain: Visual geometry, Collision geometry, Inertial properties.

Geometry

Defines the shape of an object. Examples: <box>, <cylinder>, <sphere>, <plane>, <mesh>.

Visual & Collision

- **<visual>**: Controls how an object appears.
- **<collision>**: Defines how objects interact physically. Collision geometry is often simpler than the visual geometry to improve simulation performance.

Understanding a URDF Robot Model

A robot in URDF is represented as a collection of links connected together using joints. The robot definition begins with:

```
<robot name="mini_bot">  
</robot>
```

All links, joints, sensors, and plugins belonging to the robot are defined inside this tag.

Links

A link represents a rigid body of the robot. Examples include the chassis, wheels, sensors, and other physical components.

```
<link name="base_link">  
</link>
```

Each link generally contains three important sections:

Inertial

The `<inertial>` tag defines the physical properties of the link required for simulation.

- mass defines the weight of the link.
- origin defines the position of the center of mass.
- inertia defines the inertia matrix used by the physics engine.

Visual

The `<visual>` tag controls how the link appears in visualization tools such as RViz and Gazebo.

- origin defines the position and orientation of the visual model.
- geometry defines the shape of the object.
- Mesh files (.stl, .dae, etc.) are commonly used for realistic visualization.

Collision

The `<collision>` tag defines how the link interacts with the environment during simulation.

- origin defines the collision model position.
- geometry defines the collision shape.

Collision models are often simplified compared to visual models to improve simulation performance.

Joints

A **joint** connects two links together and defines how they move relative to one another.

```
<joint name="Right_wheel_joint" type="continuous">
```

`</joint>`

Important elements inside a joint are:

- parent – the link from which the joint originates.
- child – the link attached to the joint.
- origin – position and orientation of the joint.
- axis – axis of motion.
- limit – constraints on motion such as velocity and effort.

A continuous joint behaves like a revolute joint with unlimited rotation and is commonly used for wheels.

Gazebo Plugins:-

Gazebo plugins are shared libraries that extend the functionality of a simulation. They can be attached to a world, model, sensor, or system to provide custom behavior such as robot control, sensor simulation, data publishing, and interaction with external applications.

Plugins enable robots to receive commands, publish sensor data, and perform specific tasks during simulation. For example, the **DiffDrive** plugin allows a differential drive robot to move by subscribing to velocity commands, while the **Key Publisher** and **Triggered Publisher** plugins enable keyboard-based robot control.

Differential Drive Plugin

To make a differential drive robot move inside Gazebo, we can use the `DiffDrive` plugin.

Plugin Parameters

`filename`

Specifies the shared library that contains the plugin implementation.

```
<filename>libignition-gazebo-diff-drive-system.so</filename>
```

`name`

Specifies the plugin class that will be loaded by Gazebo.

```
<name>ignition::gazebo::systems::DiffDrive</name>
```

`left_joint` and `right_joint`

Define the wheel joints controlled by the plugin.

```
<left_joint>left_wheel_joint</left_joint>
```

```
<right_joint>right_wheel_joint</right_joint>
```

`wheel_separation`

Distance between the left and right wheels.

```
<wheel_separation>1.2</wheel_separation>
```

wheel_radius

Radius of the robot wheels.

```
<wheel_radius>0.4</wheel_radius>
```

odom_publish_frequency

Frequency at which odometry information is published.

```
<odom_publish_frequency>1</odom_publish_frequency>
```

topic

Topic on which velocity commands are received.

```
<topic>cmd_vel</topic>
```

Note:- There are several other Gazebo plugins available for extending robot functionality. Commonly used plugins include camera plugins for image acquisition, IMU plugins for inertial measurements, and LiDAR plugins for range sensing. These plugins enable realistic sensor simulation and are widely used in robotics applications for perception and navigation.

You can refer to [Sensor Plugins](#) to learn more about sensor plugins. Follow [List](#) to know all types of plugin files.

Building a Complete Robot

To create a complete robot model:

1. Define the robot using the `<robot>` tag.
2. Create links for the chassis and all moving parts.
3. Add visual, collision, and inertial properties for each link.
4. Connect links using joints.
5. Add sensors, transmissions, and Gazebo plugins as required.

Following this approach, the mini_bot model is constructed using a base link and three omni wheels connected through joints, forming a complete mobile robot.

Rviz

RViz (ROS Visualization) is the official 3D visualization tool for ROS and ROS 2. It allows users to visualize robot models, sensor data, coordinate frames, maps, paths, and other information published by ROS nodes.

Unlike Gazebo, RViz does not simulate physics or robot dynamics. Instead, it provides a graphical interface for monitoring and debugging the state of a robot.

Features of RViz

RViz can be used to visualize:

- Robot models (URDF/Xacro).
- Coordinate frames (TF).

- Laser scans and point clouds.
- Camera images.
- IMU and odometry data.
- Occupancy grids and maps.
- Planned paths and navigation goals.
- Marker arrays for debugging.

Robot Visualization

The robot model is typically published using the `robot_state_publisher` node, which reads the URDF/Xacro file and publishes the transforms between the robot links. RViz then renders the robot model and updates its configuration based on the published joint states.

Launching RViz

To visualize a URDF file:

```
Shell
ros2 run robot_state_publisher robot_state_publisher mini_bot.urdf
```

In another terminal:

```
Shell
rviz2
```

Then set the frame to `base_link`. Add robot model and set topic to `robot_description`

Introduction to TF in ROS 2

What is TF?

TF (Transform Framework) is used to define and manage relationships between different coordinate frames in a robot. It helps answer questions like:

- Where is the camera relative to the robot?
- Where is the robot relative to the world?
- Where is an obstacle detected by the LiDAR in the map?

Why is TF Needed?

Every sensor works in its own coordinate frame (e.g., camera_link, laser_link, imu_link). To combine data from multiple sensors, ROS must convert coordinates between these frames. TF performs these transformations automatically. Typical TF Tree

map -> odom -> base_link -> (camera_link, laser_link, left_wheel, right_wheel) Important Frames

- **base_link**: Reference frame attached to the robot body.
- **odom**: Odometry frame generated from wheel motion. It provides the robot's estimated position but may drift over time.
- **map**: Global frame generated by SLAM/AMCL. It corrects odometry drift and is used for navigation.

Useful Commands

- **Generate TF tree**: `ros2 run tf2_tools view_frames`
- **View a transform**: `ros2 run tf2_ros tf2_echo odom base_link`
- **View TF messages**: `ros2 topic echo /tf`

Resources:-

You can also follow the ROS documentation for URDF 🖱️ [ROS Urdf](#)

Follow the official documentation of fortress 🖱️ [Fortress Documentation](#)

Follow this for Rviz 🖱️ [Rviz](#)

For making python launch file 🖱️ <https://roboticsbackend.com/ros2-launch-file-example>

(Use chatgpt to understand how to create a launch file for both RViz and Ignition Fortress)

To understand TF:- https://youtu.be/QyvHhY4Y_Y8?si=s-TQdnLSjzWDtFvV

To understand how to write urdf and launching in RViz:- [URDF Tutorial](#)

Most YouTube tutorials use Gazebo Classic, which is deprecated, while only a few cover Ignition Fortress. URDF creation is largely the same, so you can follow those tutorials for robot modeling. The main differences arise during the simulation setup like plugin, ros2 controllers and bridge setup, where Ignition Fortress requires some modifications, consultation of the official documentation or AI.

Use of AI tools is permitted for learning purposes only. .